



FGA 0314 - Testes de Software – Turma: Turma 1

Semestre: 2025.2

Nome: Gustavo Oki de Freitas Rodrigues Leite

Matrícula: 231034716

Equipe: Grupo 7 - Testus Maximus

Atividade 4 – TDD

1 Funcionalidade

1.1 Identificação da Issue

A issue escolhida para este PTOSS, é a de número 731 do repositório BrasilAPI, disponível em <https://github.com/BrasilAPI/BrasilAPI/issues/731>, intitulada "[CEP] Exceções não tratadas ao consultar CEP mal formatado".

1.2 Especificação

Esta issue identifica um erro crítico no endpoint de consulta de CEP da BrasilAPI, especificamente na versão 1 da API disponível em <https://brasilapi.com.br/api/cep/v1/{cep}>. O problema ocorre quando um usuário consulta um CEP que, apesar de atender às condições básicas de formatação (8 caracteres numéricos com hífen antes dos 2 últimos dígitos), é inválido ou inexistente. Nesta situação, o sistema retorna mensagens de erro que expõem detalhes técnicos da implementação interna, os quais não deveriam ser visíveis ao usuário final.

Para acessar o problema, é preciso fazer uma requisição GET para o endpoint <https://brasilapi.com.br/api/cep/v1/002123-25>. A resposta atual, que está incorreta, retorna um objeto JSON contendo mensagens técnicas como "A autenticao de null falhou!" do serviço dos Correios e "Cannot read properties of undefined (reading 'replace!)" do ViaCEP. Além disso, o WideNet retorna "Erro ao se conectar com o serviço WideNet" e apenas o serviço Correios-alt retorna uma mensagem apropriada informando "CEP não encontrado na base dos Correios".

O comportamento esperado, de acordo com a documentação oficial da API, seria retornar mensagens padronizadas e amigáveis para todos os serviços. O serviço dos Correios deveria retornar "CEP INVÁLIDO", o ViaCEP deveria informar "CEP não encontrado na base do ViaCEP", o WideNet deveria retornar simplesmente "CEP não encontrado" e o Correios-alt manteria sua mensagem "CEP não encontrado na base dos Correios". Todas essas mensagens seguiriam a estrutura padrão da API com o tipo de erro "service_error" e nome "CepPromiseError".

O impacto deste problema é significativo sob diversos aspectos. Primeiro, a exposição de erros internos como "Cannot read properties of undefined" e "A autenticação de null falhou!" revela detalhes técnicos da implementação que podem ser explorados maliciosamente. Segundo, prejudica a imagem e credibilidade do projeto open source, transmitindo uma impressão de falta de profissionalismo. Terceiro, causa confusão aos desenvolvedores que consomem a API, dificultando o tratamento adequado de erros em suas aplicações. Por fim, representa uma potencial vulnerabilidade de segurança conhecida como "information disclosure", onde informações sensíveis sobre a arquitetura e funcionamento interno do sistema são inadvertidamente expostas.

1.3 Descrição da Funcionalidade

A funcionalidade a ser implementada consiste em criar uma camada robusta de tratamento de exceções para o serviço de consulta de CEP da BrasilAPI. O objetivo principal é interceptar erros técnicos antes que cheguem ao usuário final e transformá-los em mensagens padronizadas, amigáveis e profissionais, seguindo rigorosamente o padrão documentado da API.

A solução envolve cinco componentes principais que trabalham em conjunto. Primeiro, é necessário implementar uma captura eficiente de exceções técnicas, interceptando erros como referências nulas, propriedades indefinidas e falhas de conexão antes que sejam expostos aos consumidores da API. Segundo, deve-se criar um mecanismo de normalização de mensagens que converta mensagens técnicas em mensagens padronizadas e compreensíveis para desenvolvedores que utilizam o serviço.

Terceiro, é essencial adicionar validações preventivas que impeçam que CEPs mal formatados ou inválidos cheguem aos serviços externos de consulta, reduzindo chamadas desnecessárias e melhorando a performance. Quarto, toda a estrutura de respostas de erro deve

ser padronizada para garantir consistência em todos os cenários possíveis, facilitando o tratamento de erros por parte dos consumidores da API.

Por fim, apesar de ocultar os detalhes técnicos dos usuários finais, é fundamental manter um sistema robusto de logging interno que registre todas as exceções detalhadas para fins de debugging e monitoramento, permitindo que a equipe de desenvolvimento identifique e corrija problemas sem comprometer a segurança ou experiência do usuário.

A implementação deve tratar de forma consistente os quatro serviços utilizados pela BrasilAPI para consulta de CEP: Correios, ViaCEP, WideNet e Correios-alt. Cada serviço possui suas peculiaridades e tipos de erros, mas todos devem seguir o mesmo padrão de resposta ao usuário final, garantindo uniformidade e previsibilidade no comportamento da API.

1.4 Ciclos

O desenvolvimento desta funcionalidade foi planejado seguindo a metodologia TDD (Test-Driven Development) através de seis ciclos incrementais, cada um focado em um aspecto específico do problema. O primeiro ciclo concentra-se em criar um teste que valide que erros técnicos do serviço ViaCEP, especificamente a mensagem "Cannot read properties of undefined", não sejam expostos aos usuários da API.

O segundo ciclo expande a cobertura para incluir validação dos erros técnicos provenientes do serviço dos Correios, garantindo que mensagens como "A autenticao de null falhou!" sejam adequadamente interceptadas e normalizadas. O terceiro ciclo amplia ainda mais o escopo, validando que todos os quatro serviços de CEP (Correios, ViaCEP, WideNet e Correios-alt) retornem mensagens padronizadas conforme especificado na documentação da API.

No quarto ciclo, são implementados testes de regressão para assegurar que a correção implementada não quebrou funcionalidades existentes, validando que CEPs válidos continuam sendo processados corretamente e retornando os dados esperados. O quinto ciclo foca em testes de integração mais abrangentes, cobrindo múltiplos formatos de CEP inválidos como aqueles com zeros à esquerda, dígitos inexistentes ou formatações não convencionais.

Finalmente, o sexto ciclo é dedicado à refatoração do código implementado, criando helpers de normalização de erros, melhorando a documentação através de comentários JSDoc,

adicionando sistema de logging interno para debugging e extraíndo constantes para facilitar a manutenção futura do código. Esta abordagem incremental garante que cada aspecto do problema seja tratado de forma sistemática e bem testada.

2 Execução

2.1 Primeiro Ciclo

<identifique o primeiro teste e apresente uma breve descrição>

<apresente o código de teste>

<apresente o resultado da execução do teste (falha)>

<apresente o código da funcionalidade>

<apresente o resultado da execução do teste (sucesso)>

<apresente o código refatorado, se for o caso>

<apresente o resultado da execução do teste (sucesso)>

Identificação: Teste para evitar exposição de erro técnico do ViaCEP

Descrição: Validar que erros técnicos como "Cannot read properties of undefined" não são expostos ao usuário.

2.2 Segundo Ciclo

<identifique o segundo teste e apresente uma breve descrição>

<apresente o código de teste>



<apresente o resultado da execução do teste (falha)>

<apresente o código da funcionalidade>

<apresente o resultado da execução do teste (sucesso)>

<apresente o código refatorado, se for o caso>

<apresente o resultado da execução do teste (sucesso)>

2.n N-ésimo Ciclo

<identifique o n-ésimo teste e apresente uma breve descrição>

<apresente o código de teste>

<apresente o resultado da execução do teste (falha)>

<apresente o código da funcionalidade>

<apresente o resultado da execução do teste (sucesso)>

<apresente o código refatorado, se for o caso>

<apresente o resultado da execução do teste (sucesso)>

3 Código Fonte Testes

<Apresentar a versão final do código fonte dos testes>

<Indicar o link para a classe no seu repositório Github (fork do projeto)>

4 Resultado Final Execução Testes

<Apresentar o resultado final da execução de todos os testes>

5 Código Fonte da Funcionalidade Implementada

<Apresentar a versão final do código fonte do funcionalidade implementada>

<Indicar o link para a classe no seu repositório Github (fork do projeto)>

6 Pull Request

<Caso faça um pull request, apresente aqui uma captura de tela do pull request e o link para o mesmo>

7 Conclusão

A experiência de aplicar TDD para resolver a issue mostrou como essa metodologia pode fortalecer a segurança e a qualidade em uma API. A issue era crítica porque mostrava detalhes técnicos da implementação aos usuários, mensagens como “Cannot read properties of undefined” e “A autenticação de null falhou!” não só confundiam desenvolvedores como podiam revelar vulnerabilidades ao expor a arquitetura interna.

Ao trabalhar com TDD, passei a pensar primeiro no que não pode acontecer e a escrever testes que verificavam a ausência dessas mensagens técnicas, criando uma mentalidade “security-first” desde o início. O desenvolvimento incremental permitiu descobrir múltiplos cenários problemáticos, sendo que a cada ciclo surgiam novos tipos de erro técnico a serem tratados e os seis ciclos realizados garantiram cobertura dos quatro serviços envolvidos (Correios, ViaCEP, WideNet e Correios-alt). Isso deu confiança para refatorar o código, acrescentado o logging no sexto ciclo, melhorando a documentação e expandindo padrões regex sem quebrar funcionalidades, porque a suíte de testes protegia a mudança. Incluir testes de



regressão para CEPs válidos foi essencial para garantir que a correção não afetasse o comportamento esperado, algo que pode ser considerado crítico em APIs públicas com muitos usuários. Os próprios testes funcionaram como documentação executável, ajudando futuros desenvolvedores para que entendam com clareza o que deve e o que não deve ser exposto. Aprendi que tratamento de erro é uma feature crítica que merece o mesmo nível de testes e atenção que outras funcionalidades, e que criar uma camada de normalização (`CepErrorHandler`) é um padrão reutilizável para outros serviços.

Para mim, ficou claro que TDD é essencial em correções de segurança, que pensar em casos extremos desde o início é crucial, que logging é parte da solução e que abstrações como o `CepErrorHandler` são reutilizáveis em outros endpoints. Testes são um investimento que se paga com menos bugs em produção, debugging mais rápido, refatorações seguras e documentação viva. Não só isso, mas foi perceptível que a aplicação do TDD transforma uma correção simples em uma melhoria notável de qualidade e segurança: uma solução bem testada, código mais limpo e manutenível, documentação executável, proteção contra regressões e um padrão replicável para outros endpoints.